

GCP Terraform + Ansible Lab — Every Step Standalone

Rule: the first line of any step that writes a file is mkdir -p. Do not start with cd. Home directory is /home/rps on this VM.

STEP 1 — Create directories (run this first, before any cd)

```
mkdir -p /home/rps/tf-ansible-pg/env
mkdir -p /home/rps/tf-ansible-pg/modules/network
mkdir -p /home/rps/tf-ansible-pg/modules/iam
mkdir -p /home/rps/tf-ansible-pg/modules/compute
mkdir -p /home/rps/tf-ansible-pg/modules/sql
mkdir -p /home/rps/tf-ansible-pg/ansible/playbooks
mkdir -p /home/rps/tf-ansible-pg/ansible/inventories
ls -la /home/rps/tf-ansible-pg
```

Expected: listing shows env, modules, ansible. If this fails, stop.

STEP 2 — Enter the lab directory

```
cd /home/rps/tf-ansible-pg
pwd
```

pwd must print /home/rps/tf-ansible-pg. If No such file or directory, you skipped STEP 1.

STEP 3 — Show GCP project

```
gcloud config get-value project
```

If empty: gcloud config set project YOUR_PROJECT_ID then repeat STEP 3.

STEP 4 — Write env/terraform.tfvars using a full path

```
mkdir -p /home/rps/tf-ansible-pg/env
PROJECT_ID=$(gcloud config get-value project)
echo "PROJECT_ID is $PROJECT_ID"
cat > /home/rps/tf-ansible-pg/env/terraform.tfvars <<TFVARS
project_id      = "${PROJECT_ID}"
region          = "asia-south1"
zone            = "asia-south1-a"
network_name    = "pg-lab-vpc"
subnet_name     = "pg-lab-subnet"
subnet_cidr     = "10.20.0.0/24"
vm_name         = "pg-ansible-vm"
sql_name        = "pg-lab-sql"
sql_password    = "ChangeMe-Lab-Only"
create_network  = false
TFVARS
cat /home/rps/tf-ansible-pg/env/terraform.tfvars
```

The project_id line must contain your real project, not a blank.

STEP 5 — Enable APIs

```
gcloud services enable compute.googleapis.com sqladmin.googleapis.com iam.googleapis.com servicenetworking.googleapis.com cloudresourcemanager.googleapis.com
```

STEP 6 — Create VPC if missing (lab 1.1)

```
gcloud compute networks describe pg-lab-vpc
```

If describe fails with NOT_FOUND, create it:

```
gcloud compute networks create pg-lab-vpc --subnet-mode=custom --bgp-routing-mode=regional
gcloud compute networks subnets describe pg-lab-subnet --region=asia-south1
```

If NOT_FOUND:

```
gcloud compute networks subnets create pg-lab-subnet --network=pg-lab-vpc --region=asia-south1 --range=10.20.0.0/24 --enable-private-ip-goog
```

STEP 7 — Firewalls if missing (lab 1.2)

```
gcloud compute firewall-rules describe pg-lab-allow-iap-ssh
```

If NOT_FOUND:

```
gcloud compute firewall-rules create pg-lab-allow-iap-ssh --network=pg-lab-vpc --allow=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=pg-lab-allow-iap-ssh
gcloud compute firewall-rules describe pg-lab-allow-5432-internal
```

If NOT_FOUND:

```
gcloud compute firewall-rules create pg-lab-allow-5432-internal --network=pg-lab-vpc --allow=tcp:5432 --source-ranges=10.20.0.0/24 --target-tags=pg-lab-allow-5432-internal
```

STEP 8 — PSA for Cloud SQL if missing (lab 1.3)

```
gcloud compute addresses describe pg-lab-psa --global
```

If NOT_FOUND:

```
gcloud compute addresses create pg-lab-psa --global --purpose=VPC_PEERING --prefix-length=16 --network=pg-lab-vpc
```

```
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com --ranges=pg-lab-psa --network=pg-lab-vpc
gcloud services vpc-peerings list --network=pg-lab-vpc
```

If connect says already exists, that is success. Do not create another PSA address.

STEP 9 — Write versions.tf

```
mkdir -p /home/rps/tf-ansible-pg
cat > /home/rps/tf-ansible-pg/versions.tf <<'EOF'
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}
provider "google" {
  project = var.project_id
  region  = var.region
}
EOF
```

STEP 10 — Write variables.tf

```
mkdir -p /home/rps/tf-ansible-pg
cat > /home/rps/tf-ansible-pg/variables.tf <<'EOF'
variable "project_id" { type = string }
variable "region" { type = string }
variable "zone" { type = string }
variable "network_name" { type = string }
variable "subnet_name" { type = string }
variable "subnet_cidr" { type = string }
variable "vm_name" { type = string }
variable "sql_name" { type = string }
variable "sql_password" { type = string; sensitive = true }
variable "create_network" { type = bool; default = false }
EOF
```

STEP 11 — Write modules/network/main.tf

```
mkdir -p /home/rps/tf-ansible-pg/modules/network
cat > /home/rps/tf-ansible-pg/modules/network/main.tf <<'EOF'
variable "create_network" { type = bool }
variable "network_name" { type = string }
variable "subnet_name" { type = string }
variable "subnet_cidr" { type = string }
variable "region" { type = string }
data "google_compute_network" "this" { name = var.network_name }
data "google_compute_subnetwork" "this" { name = var.subnet_name; region = var.region }
resource "google_compute_global_address" "psa" {
  name = "pg-lab-psa"
  purpose = "VPC_PEERING"
  address_type = "INTERNAL"
  prefix_length = 16
  network = data.google_compute_network.this.id
}
resource "google_service_networking_connection" "psa" {
  network = data.google_compute_network.this.id
  service = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.psa.name]
}
resource "google_compute_firewall" "iap_ssh" {
  name = "pg-lab-allow-iap-ssh"
  network = data.google_compute_network.this.name
  allow { protocol = "tcp"; ports = ["22"] }
  source_ranges = ["35.235.240.0/20"]
  target_tags = ["pg-lab"]
}
resource "google_compute_firewall" "pg" {
  name = "pg-lab-allow-5432-internal"
  network = data.google_compute_network.this.name
  allow { protocol = "tcp"; ports = ["5432"] }
  source_ranges = [var.subnet_cidr]
  target_tags = ["pg-lab"]
}
output "network_id" { value = data.google_compute_network.this.id }
```

```
output "subnet_self_link" { value = data.google_compute_subnetwork.this.self_link }
EOF
```

STEP 12 — Write modules/iam/main.tf

```
mkdir -p /home/rps/tf-ansible-pg/modules/iam
cat > /home/rps/tf-ansible-pg/modules/iam/main.tf <<'EOF'
variable "project_id" { type = string }
resource "google_service_account" "vm" { account_id = "pg-vm-sa"; display_name = "PostgreSQL VM" }
resource "google_service_account" "sql" { account_id = "pg-sql-client-sa"; display_name = "Cloud SQL client" }
resource "google_project_iam_member" "vm_log" {
  project = var.project_id
  role = "roles/logging.logWriter"
  member = "serviceAccount:${google_service_account.vm.email}"
}
resource "google_project_iam_member" "vm_mon" {
  project = var.project_id
  role = "roles/monitoring.metricWriter"
  member = "serviceAccount:${google_service_account.vm.email}"
}
resource "google_project_iam_member" "sql_client" {
  project = var.project_id
  role = "roles/cloudsql.client"
  member = "serviceAccount:${google_service_account.sql.email}"
}
output "vm_email" { value = google_service_account.vm.email }
output "sql_email" { value = google_service_account.sql.email }
EOF
```

STEP 13 — Write modules/compute/main.tf

```
mkdir -p /home/rps/tf-ansible-pg/modules/compute
cat > /home/rps/tf-ansible-pg/modules/compute/main.tf <<'EOF'
variable "name" { type = string }
variable "zone" { type = string }
variable "subnet" { type = string }
variable "sa_email" { type = string }
data "google_compute_image" "ubuntu" { family = "ubuntu-2404-lts-amd64"; project = "ubuntu-os-cloud" }
resource "google_compute_instance" "vm" {
  name = var.name
  machine_type = "e2-medium"
  zone = var.zone
  tags = ["pg-lab"]
  boot_disk { initialize_params { image = data.google_compute_image.ubuntu.self_link; size = 20; type = "pd-balanced" } }
  network_interface { subnetwork = var.subnet }
  service_account { email = var.sa_email; scopes = ["cloud-platform"] }
  metadata = { enable-oslogin = "TRUE" }
}
output "internal_ip" { value = google_compute_instance.vm.network_interface[0].network_ip }
EOF
```

STEP 14 — Write modules/sql/main.tf

```
mkdir -p /home/rps/tf-ansible-pg/modules/sql
cat > /home/rps/tf-ansible-pg/modules/sql/main.tf <<'EOF'
variable "name" { type = string }
variable "region" { type = string }
variable "network_id" { type = string }
variable "password" { type = string; sensitive = true }
resource "google_sql_database_instance" "this" {
  name = var.name
  database_version = "POSTGRES_16"
  region = var.region
  settings {
    tier = "db-custom-1-3840"
    ip_configuration { ipv4_enabled = false; private_network = var.network_id }
    availability_type = "ZONAL"
    disk_size = 20
  }
  root_password = var.password
}
resource "google_sql_database" "appdb" { name = "appdb"; instance = google_sql_database_instance.this.name }
resource "google_sql_user" "appuser" { name = "appuser"; instance = google_sql_database_instance.this.name; password = var.password }
output "connection_name" { value = google_sql_database_instance.this.connection_name }
output "private_ip" { value = google_sql_database_instance.this.private_ip_address }
EOF
```

STEP 15 — Write main.tf

```
mkdir -p /home/rps/tf-ansible-pg
cat > /home/rps/tf-ansible-pg/main.tf <<'EOF'
module "network" {
```

```

source = "./modules/network"
create_network = var.create_network
network_name = var.network_name
subnet_name = var.subnet_name
subnet_cidr = var.subnet_cidr
region = var.region
}
module "iam" { source = "./modules/iam"; project_id = var.project_id }
module "compute" {
  source = "./modules/compute"
  name = var.vm_name
  zone = var.zone
  subnet = module.network.subnet_self_link
  sa_email = module.iam.vm_email
}
module "sql" {
  source = "./modules/sql"
  name = var.sql_name
  region = var.region
  network_id = module.network.network_id
  password = var.sql_password
  depends_on = [module.network]
}
output "vm_ip" { value = module.compute.internal_ip }
output "sql_connection" { value = module.sql.connection_name }
output "sql_private_ip" { value = module.sql.private_ip }
EOF
ls -l /home/rps/tf-ansible-pg

```

STEP 16 — terraform init

```

cd /home/rps/tf-ansible-pg
terraform init

```

STEP 17 — Import existing 1.1–1.3 objects (ignore errors if already in state)

```

cd /home/rps/tf-ansible-pg
PROJECT_ID=$(gcloud config get-value project)
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars module.network.google_compute_global_address.psa pg-lab-psa
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars module.network.google_compute_firewall.iap_ssh pg-lab-allow-iap-ssh
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars module.network.google_compute_firewall.pg pg-lab-allow-5432-internal
terraform import -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars 'module.network.google_service_networking_connection.psa' "projects/

```

STEP 18 — Plan and apply

```

cd /home/rps/tf-ansible-pg
terraform plan -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars -out=tfplan
terraform apply tfplan
terraform output

```

STEP 19 — Ansible playbook files

```

mkdir -p /home/rps/tf-ansible-pg/ansible/playbooks
mkdir -p /home/rps/tf-ansible-pg/ansible/inventories
cd /home/rps/tf-ansible-pg
VM_IP=$(terraform output -raw vm_ip)
echo "VM_IP=$VM_IP"
cat > /home/rps/tf-ansible-pg/ansible/inventories/lab.ini <<EOF
[postgres]
pg-ansible-vm ansible_host=${VM_IP} ansible_user=rps ansible_python_interpreter=/usr/bin/python3
EOF

mkdir -p /home/rps/tf-ansible-pg/ansible/playbooks
cat > /home/rps/tf-ansible-pg/ansible/playbooks/postgres.yml <<'EOF'
---
- hosts: postgres
  become: true
  vars:
    pg_version: 16
  tasks:
    - apt:
      name: ["postgresql-{{ pg_version }}", "postgresql-contrib-{{ pg_version }}", "python3-psycopg2"]
      update_cache: true
    - lineinfile:
      path: "/etc/postgresql/{{ pg_version }}/main/postgresql.conf"
      regexp: "^#?listen_addresses"
      line: "listen_addresses = '*'
      notify: reload postgres
    - lineinfile:
      path: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf"
      line: "host all all 10.20.0.0/24 scram-sha-256"
      notify: reload postgres
    - file:

```

```

    path: /var/backups/postgresql
    state: directory
    owner: postgres
    mode: "0750"
  - become_user: postgres
    community.postgresql.postgresql_user:
      name: appuser
      password: "ChangeMe-Lab-Only"
      role_attr_flags: LOGIN
  - become_user: postgres
    community.postgresql.postgresql_db:
      name: appdb
      owner: appuser
handlers:
  - name: reload postgres
    service:
      name: postgresql
      state: reloaded
EOF

```

STEP 20 — Install PostgreSQL with Ansible on the VM

```
gcloud compute ssh pg-ansible-vm --zone=asia-south1-a --tunnel-through-iap
```

On the VM, run:

```

sudo apt-get update
sudo apt-get install -y ansible python3-psycopg2
ansible-galaxy collection install community.postgresql

```

Copy /home/rps/tf-ansible-pg/ansible/playbooks/postgres.yml onto the VM (or recreate the file there), then:

```

ansible-playbook -i localhost, -c local postgres.yml
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres pg_dump -Fc appdb -f /var/backups/postgresql/appdb.dump
ls -l /var/backups/postgresql/

```

STEP 21 — Cleanup

```

cd /home/rps/tf-ansible-pg
terraform destroy -var-file=/home/rps/tf-ansible-pg/env/terraform.tfvars

```

Errors you already hit

cd: ... No such file or directory → run STEP 1 mkdir, then STEP 2 cd.

cat: env/terraform.tfvars: No such file or directory → run STEP 1 and STEP 4 (full path).